DESAFÍO TOLOTÁ

GUÍA DE LA APLICACIÓN

Cómo está hecha la página web y el sistema de cronometraje, su modelo de datos, las pruebas que se hicieron, los cambios que sugiero y lo que hay que saber para seguir actualizándola sin desordenarla.

1. Qué es la aplicación
2. Tecnologías
3. Arquitectura y día del evento
4. Estructura de carpetas
5. Modelo de datos (entidad-relación)
6. Páginas, roles y API
7. Cómo funcionan los procesos clave
8. Pruebas realizadas
9. Cambios sugeridos
10. Cómo seguir actualizando la página
11. Comandos y rutina de cada año

Rama de trabajo: develop

12 de noviembre de 2026

01 QUÉ ES LA APLICACIÓN

Es una sola aplicación web con dos partes que comparten diseño, código y base de datos:

Sitio público

Lo ve cualquiera, sin contraseña. Presenta el Desafío y muestra la información de cada edición.

- **Inicio** `/` : portada con el evento, retos, pilares, recorrido y preguntas.
- **Desafío 2026** `/desafio-2026` : el evento del año con equipos, alineaciones, pruebas y resultados en vivo.
- **Ediciones** `/ediciones` : el registro de todos los eventos.
- **Historia, Espiritual, Recursos, Galería y Contacto.**

Sistema de cronometraje

Privado, en `/cronometro` . Pide iniciar sesión con uno de dos roles:

- **Administrador:** todo (eventos, equipos, fichas médicas, disciplina, espiritual, resultados, respaldo, contraseñas).
- **Cronometrador:** solo cronometra las pruebas que el administrador habilitó y ve sus resultados. Para el histórico necesita una contraseña adicional.

IDEA CENTRAL

Todo lo que se carga en el sistema (eventos, equipos, logos, capitanes, pruebas y tiempos) aparece solo en el sitio público. No hay que tocar código para publicar los datos de un evento nuevo.

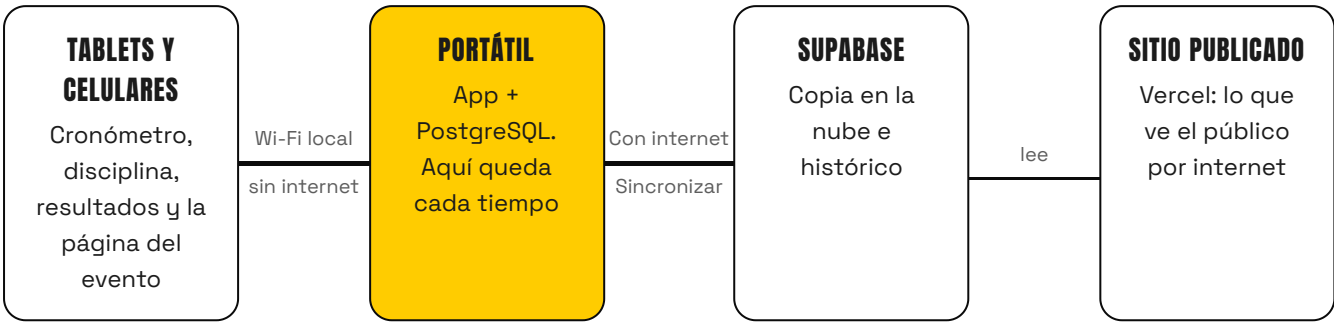
02 TECNOLOGÍAS

TECNOLOGÍA	VERSIÓN	PARA QUÉ
Next.js (App Router)	15	Sirve las páginas y la API en el mismo proyecto. Cada carpeta de `src/app` es una dirección.
React + **TypeScript**	19 / 5.9	Componentes de la interfaz. TypeScript avisa errores antes de ejecutar.
Material UI + iconos	7.3	Componentes del sistema: botones, diálogos, tablas, formularios. Estilos con `sx` y el tema (`src/theme.ts`). Por dentro usa **Emotion**.
CSS Modules	(Next)	Estilos del sitio público en archivos `.module.css` .
Fontsource	5.3	Fuentes dentro del proyecto (Anton, Space Grotesk, IBM Plex Mono): funcionan sin internet.
PostgreSQL + **Prisma**	16 / 6.19	Base de datos y la herramienta que la consulta desde el código. El modelo está en `prisma/schema.prisma` .
SWR	2.3	Pide datos a la API desde el navegador y los refresca solos (resultados en vivo).
Zod	3.25	Valida todo lo que llega a la API (tipos, tamaños, formatos).
PapaParse	5.5	Lee los archivos CSV de equipos y fichas.
Supabase	(nube)	Copia en la nube de la base de datos; ahí lee el sitio publicado.
GitHub Actions	(CI)	En cada push a `main` o `develop` : instala, migra, lint, tipos, pruebas y compila.

No se usan Tailwind, Bootstrap ni otras librerías de componentes. Colores, fuentes y efecto vidrio salen de un solo archivo: `src/lib/marca.ts` .

03 ARQUITECTURA Y DÍA DEL EVENTO

DURANTE EL EVENTO (SIN INTERNET)



- El portátil arranca con **Iniciar evento.bat**: crea la clave de sesiones si falta, prepara la base, aplica migraciones, pide las contraseñas la primera vez, compila y muestra la dirección para los demás dispositivos.
- Quienes estén en el Wi-Fi del portátil ven los resultados en vivo (cada 10 segundos).
- El sitio en internet solo se actualiza cuando se sincroniza: **Respaldo / Sincronizar ahora**. Cada sincronización reemplaza el evento completo en la nube, en una sola operación (si se corta, la nube queda como estaba).
- Las correcciones se hacen siempre en el portátil: lo que se edite directo en la nube se pierde en la siguiente sincronización.

CÓMO VIAJA UNA PETICIÓN

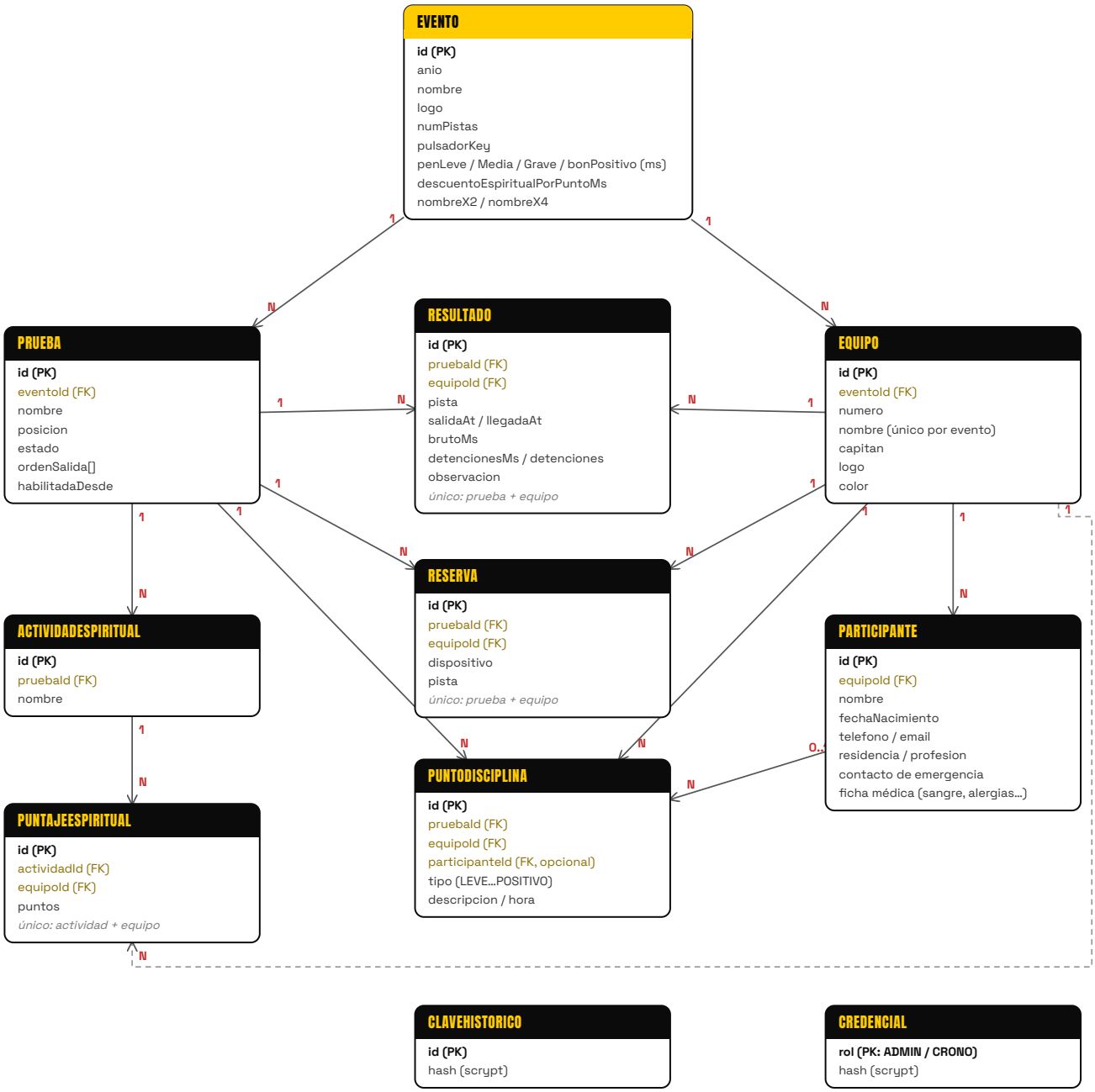
1. El navegador pide una página o la API.
2. El **middleware** (`src/middleware.ts`) revisa la cookie de sesión firmada. El sitio público y `/api/publico` pasan sin sesión; el sistema y el resto de la API la exigen. Al cronometrador solo lo deja pasar a lo permitido.
3. La ruta de la API vuelve a revisar el rol (segunda capa), valida los datos con Zod y consulta la base con Prisma.
4. La lógica pura (tiempos, posiciones, orden de salida) está en `src/lib` y tiene pruebas automáticas.

04 ESTRUCTURA DE CARPETAS

```
src/
  app/                       Páginas y API (cada carpeta = una dirección)
    page.tsx                 Portada (/)
    desafio/[anio]/          Página de cada edición (/desafio-2026)
    ediciones/ historia/ espiritual/ recursos/ galeria/ contacto/
    login/                   Inicio de sesión del sistema
    cronometro/              Sistema: lista de eventos, respaldo
      eventos/[eventoId]/    Configuración, cronometro, equipos, disciplina, espiritual,
resultados
    api/                     API (ver capítulo 6)
  components/
    landing/                 Secciones de la portada + Landing.module.css
    sitio/                   Menú, pie, marco de páginas y página de la edición (edicion/)
    crono/                   Piezas del cronómetro (pista, reloj, diálogos)
    *.tsx                    Piezas compartidas del sistema (TopBar, AppDialog, TarjetaEvento…)
  lib/                       Lógica sin pantalla
    scoring.ts               Tiempos, posiciones y general  (con pruebas)
    sesion.ts, auth.ts       Sesión firmada, roles y permisos
    validators.ts            Validación de lo que llega a la API
    respaldo.ts, nube.ts     Respaldo JSON y sincronización con Supabase
    contenido.ts             TEXTOS EDITABLES del sitio (retos, pilares, preguntas,
documentos…)
    evento.ts                Fechas, sede y ediciones del Desafío
    marca.ts                 Colores, fuentes y vidrio (todo el sitio)
    rutas.ts                 Direcciones del sitio y del sistema, y orden del menú
    landing/                 Datos de la portada, fotos, redes y página de la edición
  theme.ts, colors.ts        Tema de Material UI
  middleware.ts              Quién puede entrar a qué
prisma/                      schema.prisma (modelo) y migrations/ (cambios a las tablas)
public/fotos, public/marca   Fotos del sitio (WebP) y logo
scripts/                     Preparar base, crear secreto y contraseñas, migrar la nube
tests/                       Pruebas automáticas (npm test)
docs/                        Informes y esta guía
```

05 MODELO DE DATOS

Diagrama entidad-relación de la base de datos (PostgreSQL). **PK** es la clave principal; **FK**, la que apunta a otra tabla. **1 / N** indican la cardinalidad: un evento tiene muchas (N) pruebas.



QUÉ GUARDA CADA TABLA

TABLA	CONTENIDO Y REGLAS
Evento	Una edición del Desafío (puede haber varios por año). Pistas, tecla del pulsador, valores de disciplina y descuento espiritual, nombres de los retos X2 y X4.
Prueba	Cada reto del evento, en orden. Estado (pendiente, en curso, finalizada), orden de salida guardado y desde cuándo la puede usar el cronometrador.
Equipo	Número, nombre (único dentro del evento), capitán, logo y color de la carta del capitán.
Participante	Ficha de cada integrante: datos personales, contacto de emergencia y ficha médica. Solo la ve el administrador; el sitio público muestra nombre corto y ciudad.
Resultado	El tiempo de un equipo en una prueba (uno por equipo y prueba): salida, llegada, bruto y detenciones.
PuntoDisciplina	Penalización (leve, media, grave) o bonificación (positivo) de un equipo en una prueba; opcionalmente de un integrante.
ActividadEspiritual y **PuntajeEspiritual**	Actividades de cada prueba y los puntos de cada equipo; los puntos se convierten en descuento de tiempo.
Reserva	Qué equipo está cargado en qué pista y en qué dispositivo, para que dos tablets no carguen el mismo equipo.
Credencial y **ClaveHistorico**	Contraseñas cifradas (scrypt) de los roles y del histórico. Nunca se copian a la nube.

> ### CÓMO SE CALCULA EL TIEMPO FINAL
>
> Final = bruto − detenciones + disciplina − descuento espiritual (nunca menor que cero). Se recalcula siempre al consultar: si se registra disciplina después de la llegada, el resultado cambia solo. La general suma los finales; va primero quien completó más pruebas y, entre iguales, el menor total. Los empates comparten posición.

Al borrar un evento se borra todo lo que cuelga de él (en cascada). Para cambiar una tabla: se edita `schema.prisma`, se crea una migración en `prisma/migrations`, se aplica en local y en la nube (`npm run nube:migrar`).

06 PÁGINAS, ROLES Y API

QUIÉN VE QUÉ

PARTE	PÚBLICO	CRONOMETRADOR	ADMINISTRADOR
Sitio público y `/api/publico`	Sí	Sí	Sí
Lista de eventos, cronómetro y resultados de pruebas habilitadas	No	Sí	Sí
Histórico y exportar resultados	No	Con contraseña adicional (2 h)	Sí
Configuración, equipos y fichas, disciplina, espiritual, respaldo, nube, contraseñas	No	No	Sí

API

MÉTODOS	RUTA	USO
POST · GET	`/api/auth/login` · `/sesion` · `/estado` · `/logout`	Iniciar y cerrar sesión, saber el rol
GET PUT	`/api/auth/claves` · `/historico` · `/historico-clave`	Contraseñas de roles y del histórico
GET POST	`/api/eventos`	Listar y crear eventos (con pruebas y copia de equipos)
GET PATCH DELETE	`/api/eventos/:id`	Ver, editar y borrar un evento
POST PUT	`/api/eventos/:id/pruebas`	Agregar y reordenar pruebas
PATCH DELETE	`/api/pruebas/:id`	Estado, orden de salida, habilitación
GET POST · POST	`/api/eventos/:id/equipos` · `/equipos/import`	Equipos y su importación CSV con fichas
PATCH DELETE · POST	`/api/equipos/:id` · `/participantes`	Editar equipo, crear fichas
PATCH DELETE	`/api/participantes/:id`	Editar o borrar una ficha
GET PUT · POST	`/api/pruebas/:id/resultados` · `/intercambiar`	Guardar tiempos; corregir pista
PUT	`/api/pruebas/:id/reservas`	Equipos cargados en las pistas de cada dispositivo
DELETE	`/api/resultados/:id`	Borrar un tiempo (repetir)

GET POST · DELETE	`/api/eventos/:id/disciplina` · `/api/disciplina/:id`	Puntos de disciplina
GET POST · PATCH DELETE · PUT	`/api/pruebas/:id/actividades-espirituales` · `/api/actividades-espirituales/:id` · `/puntajes`	Actividades y puntajes espirituales
GET	`/api/eventos/:id/clasificacion` · `/exportar`	Resultados y general; CSV
GET	`/api/publico/resultados/:id`	Resultados para el sitio público (sin sesión)
GET POST	`/api/respaldo` · `/api/sincronizar`	Respaldo JSON e importación; copia a la nube

07 CÓMO FUNCIONAN LOS PROCESOS CLAVE

CRONOMETRAR UNA PRUEBA

1. Cada pista recibe el siguiente equipo del orden de salida (automático o a mano). La reserva evita que otra tablet cargue el mismo equipo.
2. **Dar salida** (botón, tecla 1–8, pulsador o encoder). **Marcar detenido** pausa y suma detenciones.
3. **LLEGADA** detiene el reloj. En el diálogo, **Guardar resultado** lo envía al servidor y aparece en Resultados; después, **Liberar pista** (o liberar y cargar el siguiente).
4. Si se recarga la página, las pistas se recuperan en ese dispositivo. Los tiempos se miden con un reloj que no cambia si la tablet ajusta su hora.

CARGAR EQUIPOS CON CSV

- Una fila por participante con su ficha (columnas como Equipo, Participante, Nacimiento, Teléfono, Residencia, Sangre, Alergias…). La plantilla está en **Equipos / Importar CSV / Plantilla**, y el archivo de **Fichas (CSV)** se puede volver a importar.
- Los equipos quedan numerados en el orden del archivo. Lo que ya existe se completa; los campos vacíos no borran nada.
- Acepta archivos de Excel en español (separados por `;`, tildes en Windows-1252, fechas dd/mm/aaaa).

PÁGINA PÚBLICA DE CADA EDICIÓN

- `/desafio-AAAA` se arma sola con el evento de ese año: portada (solo el año actual), equipos con su alineación, pruebas y resultados en vivo.
- De cada integrante solo sale primer nombre, primer apellido (con mayúscula inicial) y ciudad. El capitán va primero y su carta toma el color elegido en el sistema o, si no hay, el color más llamativo del logo.
- Las páginas se regeneran cada 5 minutos; los resultados, cada 10 segundos.

08 PRUEBAS REALIZADAS

PRUEBAS AUTOMÁTICAS (34, TODAS PASAN)

Se ejecutan con `npm test` y en GitHub en cada push. Cubren la lógica que no depende de la pantalla:

ÁREA	QUÉ SE COMPRUEBA
Tiempos y clasificación	Formato de tiempo, valor de los puntos de disciplina, detenciones, empates que comparten posición, general (más pruebas primero y luego menor total), el tiempo nunca negativo, descuento espiritual solo al equipo y prueba asignados.
Orden de salida	Patrones de orden, orden aleatorio como permutación, equipos nuevos al final, hoja imprimible con HTML escapado.
Cronómetro	Corregir pista (intercambio de equipos y tiempos), asignación automática a pistas libres, botones del encoder, vueltas del reloj, reservas por dispositivo.
Seguridad	Token de sesión válido, alterado y vencido; permisos del cronometrador; límite de 5 intentos con bloqueo de 5 minutos.
Importación CSV	Fichas con `;` y fechas de Excel, formato anterior, plantilla reimportable, tildes en Windows-1252, numeración en el orden del archivo (y números repetidos ignorados).
Sitio	Rutas públicas y privadas, la edición actual es la primera sección del menú, cuenta regresiva y ediciones, contenido (21 equipos únicos, cronograma de 6 días, retos, pilares, preguntas), nombres cortos y con mayúscula inicial.

REVISIONES ADICIONALES

- [x] **Tipos** (`npm run typecheck`) y **lint** (`npm run lint`) sin errores ni advertencias.
- [x] **Navegador:** todas las páginas públicas responden (200); `/cronometro` lleva al inicio de sesión; la API privada pide sesión (401); la API pública de resultados funciona sin sesión; el servidor no registró errores.
- [x] **Visual con datos reales:** página `/desafio-2026` con los 22 equipos en orden, alineación de Pérgamo con el capitán en morado, Ediciones, y las secciones de la portada en computador y celular.
- [x] **Datos:** los 22 capitanes coinciden con un integrante de su equipo; el cálculo de color se probó con los 22 logos reales.

LO QUE FALTA PROBAR

- [ ] Todo el sistema con sesión iniciada en pantalla (no se pudo: las contraseñas solo las escribe el organizador).
- [ ] Un **ensayo general**: portátil con `Iniciar evento.bat`, dos tablets por Wi-Fi, pulsador, una prueba completa con disciplina y espiritual, respaldo y sincronización.

09 CAMBIOS SUGERIDOS

PRIORIDAD	SUGERENCIA
ALTA	**Migrar la nube** antes de sincronizar: `npm run nube:migrar`. Supabase aún no tiene las columnas y tablas nuevas (color, reservas, espiritual).
ALTA	**Ensayo general** del día del evento con el equipo de cronometraje.
ALTA	**Subir los cambios a GitHub** y, cuando estén probados, pasarlos de `develop` a `main`.
MEDIA	**Cerrar sesiones al cambiar una contraseña.** Hoy una sesión dura 12 horas aunque se cambie la clave. Requiere que la revisión de sesión consulte la base.
MEDIA	**Contenido pendiente:** enlaces de Instagram, Facebook y WhatsApp, correo, PDFs de manuales, formulario de inscripción, nombres de los retos X2 y X4 de 2026.
MEDIA	**Páginas Historia, Espiritual y Galería** (hoy dicen «próximamente»).
MEDIA	**Limpiar datos:** el integrante «Pendiente por reasignar» de Pérgamo y capitanes escritos en minúscula.
BAJA	**Fechas y sede en el formulario del evento** (hoy están en `lib/evento.ts`), para no tocar código cada año.
BAJA	**Sincronización automática** cada pocos minutos si el portátil tiene internet durante el evento.
BAJA	**Pruebas de pantalla** (que abran el navegador y recorran el cronómetro) y separar la página del cronómetro en componentes más pequeños.
BAJA	**Historial de cambios:** quién borró o corrigió un tiempo y cuándo.

10 CÓMO SEGUIR ACTUALIZANDO LA PÁGINA

DÓNDE SE CAMBIA CADA COSA

QUIERO CAMBIAR…	DÓNDE
Equipos, logos, capitanes, integrantes, pruebas, color del capitán, nombres X2/X4	En el **sistema** (no en código). Aparecen solos en el sitio.
Textos de retos, pilares, preguntas, modalidades, documentos, inscripción, contacto	`src/lib/contenido.ts`
Fechas, sede, lugar, tema bíblico y ediciones pasadas	`src/lib/evento.ts`
Redes sociales	`src/lib/landing/redes.ts`
Fotos de la portada	Archivo en `public/fotos/` (WebP, unos 1920 px de ancho) y su ruta en `src/lib/landing/fotos.ts`
Colores, fuentes, efecto vidrio	`src/lib/marca.ts` (cambia en todo el sitio y el sistema)
Orden y nombres del menú	`src/lib/rutas.ts` (`SECCIONES`)
Diseño de una sección de la portada	Su archivo en `src/components/landing/secciones/` y sus clases en `Landing.module.css`

REGLAS PARA QUE QUEDE BIEN ESTRUCTURADA

- **Datos en el sistema, textos en `contenido.ts`, diseño en los componentes.** No escribir textos sueltos dentro del diseño.
- **Una sección nueva del sitio:** crear la carpeta en `src/app/` con `page.tsx`, usar `PaginaSitio` (trae menú, cabecera y pie) y agregarla a `SECCIONES` en `rutas.ts`.
- **Estilos:** sitio público con `.module.css` y las variables de color (`var(--destacado)` , `var(--texto)` …); sistema con Material UI (`sx`). No poner colores sueltos: usar los de `marca.ts` .
- **Estilo visual:** negro, blanco y amarillo `#FFCC00` ; Anton para títulos en mayúscula, Space Grotesk para texto, Plex Mono para tiempos; botones redondeados; sin flechas ni emojis.
- **Lógica en `src/lib`** , sin duplicar, y con una prueba en `tests/` cuando calcule algo.
- **Privacidad:** nada de la ficha (edad, teléfonos, salud) en el sitio público.
- **Antes de guardar en Git:** `npm run typecheck` , `npm run lint` y `npm test` sin errores.
- **Ramas:** trabajar en `develop` ; pasar a `main` cuando esté probado. Los commits van sin crédito a Claude.
- **No borrar `.next` ni correr `next build`** mientras el servidor de desarrollo está abierto.

11 COMANDOS Y RUTINA DE CADA AÑO

COMANDOS ÚTILES

En Windows, usar `npm.cmd` si PowerShell bloquea `npm`.

COMANDO	PARA QUÉ
`npm run dev`	Servidor de desarrollo en http://localhost:3000 (abrir uno solo)
`npm test`	Pruebas automáticas
`npm run typecheck` · `npm run lint`	Revisar tipos y estilo de código
`npx prisma migrate deploy`	Aplicar migraciones a la base local
`npm run nube:migrar`	Aplicar migraciones en Supabase (cada vez que cambie el modelo)
`npm run claves`	Crear o cambiar las contraseñas desde la terminal
`npm run db:studio`	Ver y editar la base en el navegador
Iniciar evento.bat	Arrancar el servidor del evento en el portátil

RUTINA DE CADA AÑO

1. Actualizar fechas, sede y tema en `src/lib/evento.ts` (con eso cambian el menú y la cuenta regresiva).
2. Crear el evento en el sistema: logo, pistas, valores de disciplina, nombres X2 y X4, pruebas.
3. Importar los equipos con el CSV de fichas; revisar capitanes, logos y colores.
4. Comprobar `/desafio-AAAA` y Ediciones; ajustar textos en `contenido.ts` si hace falta.
5. Si cambió el modelo: `npm run nube:migrar`. Luego **Sincronizar ahora** para publicar.
6. Ensayo general antes del evento; durante el evento, sincronizar cada vez que haya conexión.